Answer Box #1:

Talk to your neighbor(s): In PA1 what Java feature did you learn about from the A1 implementation of your ValidName design? 3 features!

Regex
Regular Expressions
matching/searching string patterns

Builtin Java library
for opening new windows
on desktop
AWT
Swing

Announcements:

PA1 grades - working on them! Resubmission info will come w/grades. Thanks for your patience!

Exam grades: We will publish 4/3/2/1 cutoffs this week + send out grade summaries!

interfaces - what's an interface?

```
interface ImageQuery {
    boolean matches(ImageData im);
}
```

type of records that have matches method that takes ImageData returns boolean.

```
record HasExtension(String ext) implements ImageQuery {
public boolean matches(ImageData im) { ... }

}

record LargerThan(int width, int height) implements ImageQuery {
public boolean matches(ImageData im) { ... }

}

ImageQuery q1 = new LargerThan(200, 400);
```

storing LargerThan in ImageQuery enabled by implements

Regions

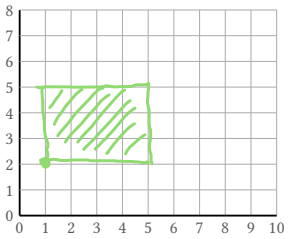
Name: _____ Date: _____

A drawing program, a map, or a game has to answer one question constantly: is this point inside this shape? Here every shape is a `Region` that can answer it. A rectangle's `corner` is its bottom-left corner.

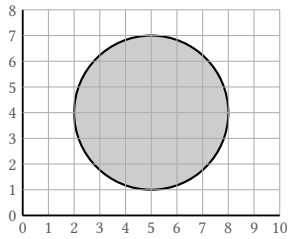
```
record Point(double x, double y) {}
interface Region { boolean contains(Point p); }
record RectRegion(Point corner, double width, double height) implements Region {
  public boolean contains(Point p) { ... }
}
record CircRegion(Point center, double radius) implements Region {
  public boolean contains(Point p) { ... }
}
```

default Region union(Region other) {
return new Union(this, other);
}

Q1. Shade `new RectRegion(new Point(1, 2), 4, 3)`.



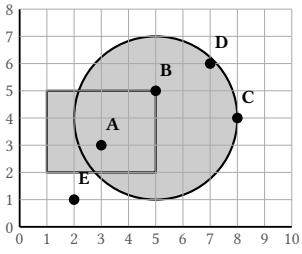
Q2. Construct this region.



new CircRegion(new Point(5, 4), 3)

From here on, `box` and `ball` are the regions from Q1 and Q2.

Q3. Which points does each region contain? Decide by looking. Leave the last column for later.



point	box.contains	ball.contains	new Union(box, ball) .contains
A (3, 3)	true	true	
B (5, 5)	true	true	
C (8, 4)	false	true	
D (7, 6)	false	true	
E (2, 1)	false	false	

Q4. Two of the points sit exactly on an edge. Which? Does a region contain its own edge? Decide; your code below makes it official.

Q5. Write the two `contains` methods.

Answer Box #2

CircRegion

```
// get distance center <-> p
// ask distance <= radius
```

RectRegion

```
boolean bwX = p.x >= corner.x &&
              p.x <= (corner.x + width);
boolean bwY = p.y >= corner.y &&
              p.y <= (corner.y + height);
return bwX && bwY;
```

Q6. A point is in `new Union(box, ball)` when it is in either one. Finish this record, then fill in the last column of Q3.

```
record Union(RectRegion a, CircRegion b) {
  public boolean contains(Point p) {
    return a.contains(p) || b.contains(p);
  }
}
```

this.a.contains(p) || this.b.contains(p)

Here are three more regions:

```
Region one = new CircRegion(new Point(4, 5), 2.5);
Region two = new CircRegion(new Point(6, 5), 2.5);
Region three = new CircRegion(new Point(5, 3), 2.5);
```

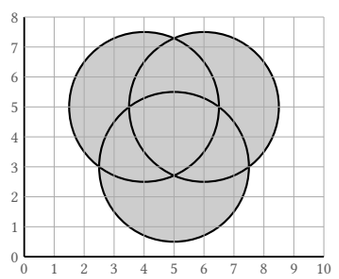
Q7. Can the `Union` from Q6 combine `one` and `two`? What does Java say? Rewrite the first line of `Union` so it can.

new Union(one, two)

CircRegion, RectRegion expected

```
record Union(Region a, Region b)
```

Q8. Construct this region, from `one`, `two`, and `three`. A `Union` holds only two regions. What else has to change in `Union` for your answer to compile?



(add implements Region to Union!)

```
Region u1 = new Union(one, two);
Region all = new Union(u1, three);
```

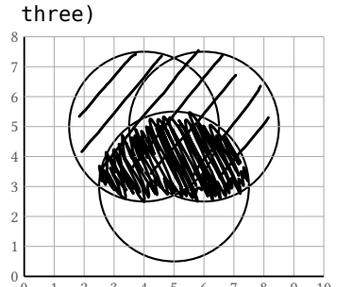
↳ without implements Region, this would error

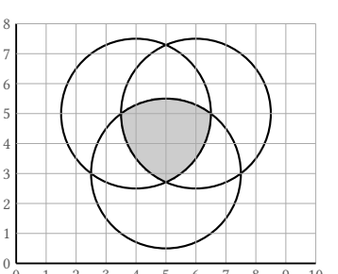
Q9. A point is in `new Intersect(a, b)` when it is in both `a` and `b`. Write the whole `Intersect` record.

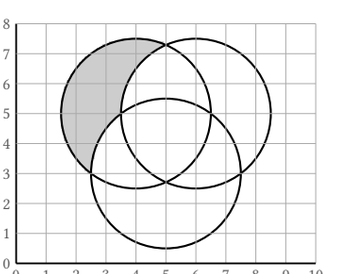
```
record Intersect(Region a, Region b) implements Region {
    public boolean contains(Point p) {
        return a.contains(p) && b.contains(p);
    }
}
```

Q10. Shade the first region. Construct the other two.

```
new Intersect(new Union(one, two),
  three)
```







```
new Intersect(
    new Intersect(one, two),
    three)
```

Q11. The last one needs a new kind of region, `Negate`: a point is in `new Negate(r)` when it is **not** in `r`. Write the `Negate` record.

```
record Negate(Region r) implements {
    public boolean contains(Point p) { return !r.contains(p); }
}
```